

Hour 3 — Styling & Component Communication

Duration

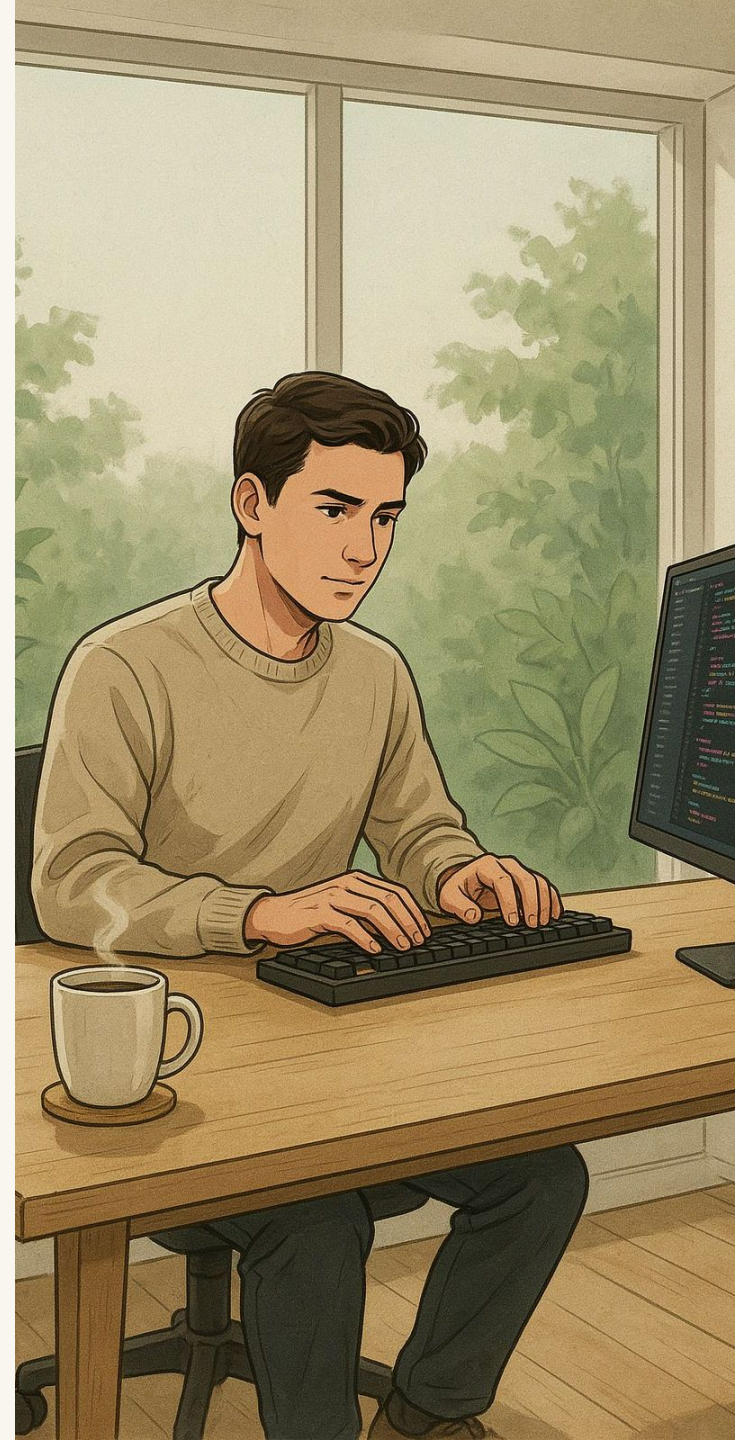
60 minutes

Topics

Styling React Components — 25 min
| Component Communication — 35 min

Level

Beginner to Lower-Intermediate



Learning Objectives

By the end of this session, learners will be able to:

Apply CSS to React components

Use plain CSS, inline styles, and CSS Modules

Understand when to use each styling approach

Pass data from parent to child using props

Send data from child to parent using callback props

Share state between sibling components using **lifting state up**

Understand the basic purpose of React Context

📄 **Props move data down. Callback functions allow events/data to move up. Shared state belongs to the nearest common parent.**

Styling React Components

React supports several ways to style components.

Three Common Approaches

Approach	Example	Best For
CSS File	StudentCard.css	Simple applications
Inline Styles	style={{...}}	Small/dynamic styles
CSS Modules	StudentCard.module.css	Component-scoped styling

Example

```
function StudentCard() {  
  return (  
    <div className="student-card">  
      <h3>Rahul</h3>  
      <p>React Developer</p>  
    </div>  
  );  
}
```

The React-specific point to remember: `className` instead of: `class`

Option 1: Plain CSS Files

StudentCard.jsx

```
import './StudentCard.css';
function StudentCard() {
  return (
    <div className="student-card">
      <h3>Rahul</h3>
      <p>React Developer</p>
    </div>
  );
}
export default StudentCard;
```

Advantages

- Easy to learn
- Familiar CSS syntax
- Good for small applications

StudentCard.css

```
.student-card {
  border: 1px solid #ddd;
  padding: 20px;
  border-radius: 8px;
}
.student-card h3 {
  margin-bottom: 5px;
}
```

Limitation

Class names are **global**. `.card { }` may accidentally conflict with another component using: `.card { }`

Option 2: Inline Styles

Styles can be defined directly inside the component.

```
function Badge() {  
  const style = {  
    backgroundColor: '#0F6E56',  
    color: 'white',  
    padding: '4px 8px',  
    borderRadius: '4px'  
  };  
  return <span style={style}>Active</span>;  
}
```

Important Difference

CSS: `background-color`

React JavaScript object: `backgroundColor`

Another Example

```
<span style={{ color: 'green', fontWeight: 'bold' }} >  
  Active  
</span>
```

Good For

- Small styles
- Dynamic styles
- One-off styling

Avoid

Using large inline style objects throughout a complex application.

Option 3: CSS Modules

CSS Modules provide **component-scoped** CSS.

StudentCard.module.css

```
.card {  
  border: 1px solid #ddd;  
  padding: 20px;  
}  
.title {  
  color: #333;  
}
```

StudentCard.jsx

```
import styles from './StudentCard.module.css';  
function StudentCard() {  
  return (  
    <div className={styles.card}>  
      <h3 className={styles.title}>  
        Rahul  
      </h3>  
    </div>  
  );  
}
```

React/build tools generate unique class names. Conceptually: `.card` ↓ `StudentCard_card_x7f82`

Benefit

Another component can also have: `.card { }` without creating a global collision.

Recommendation

For larger React applications, **CSS Modules** are a useful **component-level styling approach**.

Parent → Child Communication

The most common communication pattern is:

```
Parent
|
| props ↓
Child
```

The parent owns the data and passes it to the child.

Parent

```
function App() {
  const student = { name: 'Rahul', course: 'React' };
  return <StudentCard student={student} />;
}
```

Child

```
function StudentCard({ student }) {
  return (
    <div>
      <h3>{student.name}</h3>
      <p>{student.course}</p>
    </div>
  );
}
```

Key Point

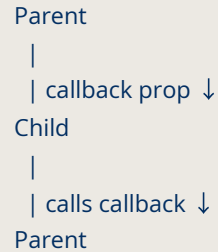
Props are:

- Passed from parent to child
- Read-only from the child's perspective
- Used to customize reusable components

Child → Parent Communication

React does not directly send data from child to parent.

Instead: The parent passes a **function** to the child.



Parent

```
function App() {  
  function handleStudent(name) {  
    console.log(name);  
  }  
  return (  
    <StudentForm onAddStudent={handleStudent} />  
  );  
}
```

Child

```
function StudentForm({ onAddStudent }) {  
  function handleSubmit() {  
    onAddStudent('Rahul');  
  }  
  return (  
    <button onClick={handleSubmit}>  
      Add Student  
    </button>  
  );  
}
```

📌 A callback prop is simply a **JavaScript function passed as a prop**.

Lifting State Up

Consider two components:

```
App
/\
StudentForm StudentList
```

Both need access to the same student data.

Instead of maintaining separate state:

```
StudentForm → students
StudentList → students
```

move the state to their common parent:

```
App
| students state
/\
StudentForm StudentList
```

This is called:

Lifting State Up

Parent

```
const [students, setStudents] = useState([]);
```

Pass data down

```
<StudentList students={students} />
```

Pass callback down

```
<StudentForm onAddStudent={addStudent} />
```

Complete Communication Example

```
function App() {  
  const [students, setStudents] = useState([]);  
  function addStudent(name) {  
    setStudents([...students, name]);  
  }  
  return (  
    <div>  
      <StudentForm onAddStudent={addStudent} />  
      <StudentList students={students} />  
      <p>Total: {students.length}</p>  
    </div>  
  );  
}
```

Data Flow

```
App  
| students state  
/\   
↓   ↓  
StudentForm StudentList  
↑   ↑  
callback  data
```

- ❑ **State should live where it is needed.** If multiple sibling components need the same state: Move the state to their nearest common parent.

Sibling Communication

React does not normally communicate directly:

Component A $\longleftrightarrow$ Component B

Instead:

```
Parent
/ \
A  B
```

The parent manages the shared state.

Example

```
SearchBox
|
| search text ↓
App
| ↓
StudentList
```

App owns:

```
const [search, setSearch] = useState("");
```

Then:

```
<SearchBox search={search} onSearch={setSearch} />
<StudentList search={search} />
```

☐ Sibling components communicate **through their parent**.

Prop Drilling & Context API

With deeply nested components, props can become repetitive.

```
App
↓
Dashboard
↓
Layout
↓
Sidebar
↓
UserProfile
```

You may have to pass:

```
user user user user
```

through components that don't actually need it.

This is called **prop drilling**.

Context API

React Context can make commonly required data available to components deeper in the tree.

Examples:

- Logged-in user
- Theme
- Language
- Application settings

For now, remember:

Props

normal component communication

Context

useful when data must be shared deeply